01NAEX - Lecture 04
Latin Squares, Graeco-Latin Squares and Balanced Incomplete Block Designs

Jiri Franc

Czech Technical University
Faculty of Nuclear Sciences and Physical Engineering
Department of Mathematics

More Blocking Designs

D. C. Montgomery, Design and Analysis of Experiments: Sections 4.2–4.4

Last lecture: the randomized complete block design (RCBD) removes the variability of **one** known and controllable nuisance factor from the experimental error.

Today: what if

- there are **two** nuisance factors (batch and operator, day and position)?
  $\Rightarrow$ **Latin square** design;
- there are **three** nuisance factors?
  $\Rightarrow$ **Graeco-Latin square** design;
- a block is too small to hold all treatments (a batch is enough for 3 runs only, a car can be tested with 4 additives only)?
  $\Rightarrow$ **balanced incomplete block design** (BIBD).

At the end: **Team Project 1** (pendulum experiment), in which you design, run and analyse your own experiment twice: as a factorial in blocks and as a Graeco-Latin square.

The Latin Square Design

A $p \times p$ Latin square is a square of p rows and p columns; each of the p^2 cells contains one of p letters (treatments), and **each letter occurs once and only once in each row and each column**:

4×4	5×5	6×6
A B D C	A D B E C	A D C E B F
B C A D	D A C B E	B A E C F D
C D B A	C B E D A	C E D F A B
D A C B	B E A C D	D C F B E A
	E C D A B	F B A D C E
		E F B A D C

- Rows and columns are two blocking factors (two restrictions on randomization): the design blocks in two directions.
- Rows, columns and treatments are mutually orthogonal, so their sums of squares separate exactly.
- Only p^2 runs for three factors with p levels each; the price is the assumption of **no interactions** between rows, columns and treatments.

A Sudoku grid is a 9×9 Latin square with an extra condition on the 3×3 boxes.

Example: Rocket Propellant

Montgomery, Example 4.3

Five formulations of a rocket propellant (A–E) are compared by their burning rate. Each batch of raw material is large enough for five formulations only, and the formulations are prepared by five operators who may differ in skill. Both batches and operators are nuisance factors: each formulation is tested exactly once in each batch and exactly once by each operator.

Latin Square Design for the Rocket Propellant Problem

Batches of Raw Material	Operators				
	1	2	3	4	5
1	$A = 24$	$B = 20$	$C = 19$	$D = 24$	$E = 24$
2	$B = 17$	$C = 24$	$D = 30$	$E = 27$	$A = 36$
3	$C = 18$	$D = 38$	$E = 26$	$A = 27$	$B = 21$
4	$D = 26$	$E = 31$	$A = 26$	$B = 23$	$C = 22$
5	$E = 22$	$A = 30$	$B = 20$	$C = 29$	$D = 31$

Rows = batches, columns = operators, letters = formulations; $N = p^2 = 25$ runs.

Latin Square - Model and ANOVA

Effects model (i = row, j = treatment, k = column; only two indices are needed to identify a cell):

$$y_{ijk} = \mu + \alpha_i + \tau_j + \beta_k + \varepsilon_{ijk}, \qquad i, j, k = 1, \ldots, p, \qquad \varepsilon_{ijk} \sim N(0, \sigma^2),$$

$\sum \alpha_i = \sum \tau_j = \sum \beta_k = 0$, no interactions. $SS_T = SS_{Rows} + SS_{Columns} + SS_{Treatments} + SS_E$.

Source	df	Sum of squares	F_0
Treatments	$p-1$	$\frac{1}{p}\sum_j y_{.j.}^2 - \frac{y_{...}^2}{N}$	$MS_{Treatments}/MS_E$
Rows	$p-1$	$\frac{1}{p}\sum_i y_{i..}^2 - \frac{y_{...}^2}{N}$	
Columns	$p-1$	$\frac{1}{p}\sum_k y_{..k}^2 - \frac{y_{...}^2}{N}$	
Error	$(p-2)(p-1)$	by subtraction	
Total	p^2-1	$\sum\sum\sum y_{ijk}^2 - \frac{y_{...}^2}{N}$	

Small squares leave few error df: 3×3 has 2, 4×4 has 6, 5×5 has 12. The analysis is a direct extension of the RCBD (Lecture 3).

Constructing and Randomizing a Latin Square

Standard Latin square: first row and first column in alphabetical order (e.g. a cyclic square, each row shifted by one).

Number of Latin squares grows very fast:

p	3	4	5	6	7
standard squares	1	4	56	9 408	16 942 080
all squares	12	576	161 280	812 851 200	$\approx 6.1 \cdot 10^{13}$

Randomization in practice:

1. Choose a standard $p \times p$ square (ideally at random among the standard squares).
2. Randomly permute the rows, the columns and the treatment labels, independently. Orthogonality and balance are preserved.
3. Map rows and columns to the nuisance factors (e.g. row = day, column = operator); if a row is a day, the runs within the day are done in random order.

Q: Which mapping (row = day, column = operator or the reverse) is easier in your lab, and does it matter for the analysis?

Rocket Propellant - ANOVA in Python

Data: `rocket.txt` (columns `y`, `batch`, `op`, `treat`).

```
URL = ("https://raw.githubusercontent.com/francji1/01NAEX/"
       "main/data/")
rocket = pd.read_csv(URL + "rocket.txt", sep=r"\s+")
ls = smf.ols("y ~ C(batch) + C(op) + C(treat)",
             data=rocket).fit()
sm.stats.anova_lm(ls, typ=2)
            sum_sq    df        F    PR(>F)
C(batch)      68.0   4.0   1.594    0.2391
C(op)        150.0   4.0   3.516    0.0404
C(treat)     330.0   4.0   7.734    0.0025
Residual     128.0  12.0
```

- The formulations differ significantly in mean burning rate ($F_0 = 7.73$, $p = 0.0025$).
- There is some evidence of differences between operators ($p = 0.040$).
- No strong evidence of differences between batches ($p = 0.24$). Blocking on batches is still a good precaution: we did not know it in advance.

Rocket Propellant - Which Formulations Differ?

Tukey's HSD with $MS_E = 10.67$ and $df_E = 12$ of the Latin square model ($n = p = 5$ runs per formulation):

```
means = rocket.groupby("treat").y.mean()
# A 28.6   B 20.2   C 22.4   D 29.8   E 26.0
q = stats.studentized_range.ppf(0.95, 5, ls.df_resid)   # 4.508
hsd = q * np.sqrt(ls.mse_resid / 5)                     # 6.58
```

- $D - B = 9.6$, $A - B = 8.4$ and $D - C = 7.4$ exceed 6.58: formulations D and A burn faster than B, and D faster than C. The other differences are not significant.
- As in Lecture 3, `pairwise_tukeyhsd()` would fit a one-way model and put the operator and batch variability into the error.

Ignoring the batches (an RCBD with operators as blocks):

```
smf.ols("y ~ C(op) + C(treat)", data=rocket).fit()
# Residual SS 196.0 on 16 df, MS_E = 12.25, treat F = 6.73
```

Here the batch blocking cost 4 error df and reduced MS_E only from 12.25 to 10.67.

Replicated Latin Squares

A small square gives few error df, so we often run n replicates (squares). The ANOVA depends on how the blocks are replicated (Montgomery, Tables 4.14–4.16): **Case 1** the same rows and columns in each replicate, **Case 2** new rows but the same columns (e.g. new batches, same operators), **Case 3** new rows and new columns.

df	Case 1	Case 2	Case 3
Treatments	$p-1$	$p-1$	$p-1$
Rows	$p-1$	$n(p-1)$	$n(p-1)$
Columns	$p-1$	$p-1$	$n(p-1)$
Replicates	$n-1$	$n-1$	$n-1$
Error	$(p-1)[n(p+1)-3]$	$(p-1)(np-2)$	$(p-1)[n(p-1)-1]$
Total	np^2-1	np^2-1	np^2-1

In Cases 2 and 3 the new rows (columns) are nested in the replicates: their SS measure the variation **within** replicates.

Crossover designs are Latin squares in which the rows are time periods and the columns are subjects; each subject receives every treatment in turn, with a washout period against carry-over effects.

Q: Would you use the same Latin square in each replicate or a newly randomized one? Why?

The Graeco-Latin Square Design

Superimpose two $p \times p$ Latin squares, one with Latin and one with Greek letters, such that **each Greek letter appears exactly once with each Latin letter**. The two squares are **orthogonal**.

4×4 Graeco-Latin Square Design

	Column			
Row	**1**	**2**	**3**	**4**
1	$A\alpha$	$B\beta$	$C\gamma$	$D\delta$
2	$B\delta$	$A\gamma$	$D\beta$	$C\alpha$
3	$C\beta$	$D\alpha$	$A\delta$	$B\gamma$
4	$D\gamma$	$C\delta$	$B\alpha$	$A\beta$

- Controls three nuisance factors (rows, columns, Greek letters), or studies two treatment factors with two blocking factors, in p^2 runs.
- Exists for every $p \geq 3$ except $p = 6$ (and not for $p = 2$). Euler's *36 officers problem* (6 ranks $\times$ 6 regiments) has no solution; this was proved by Tarry in 1901.
- All interactions are assumed negligible. Error df: $(p-3)(p-1)$, i.e. only 4 for $p = 4$ and 8 for $p = 5$.

Graeco-Latin Square - Rocket Propellant with Test Assemblies

Montgomery, Example 4.4

Suppose the five test assemblies ($\alpha, \ldots, \varepsilon$) could also matter. In Montgomery's Table 4.20 the assembly in batch i and operator j is the Greek letter number $(i + 2j) \bmod 5$ (zero-based indices); the burning rates are the same as before.

```
greek = ["alpha", "beta", "gamma", "delta", "epsilon"]
rocket["assembly"] = [greek[(b - 1 + 2 * (o - 1)) % 5]
                      for b, o in zip(rocket.batch, rocket.op)]
gl = smf.ols("y ~ C(batch) + C(op) + C(treat)"
             " + C(assembly)", data=rocket).fit()
sm.stats.anova_lm(gl, typ=2)
               sum_sq   df         F    PR(>F)
C(batch)         68.0  4.0     2.061    0.1783
C(op)           150.0  4.0     4.545    0.0329
C(treat)        330.0  4.0    10.000    0.0033
C(assembly)      62.0  4.0     1.879    0.2076
Residual         66.0  8.0
```

Removing the assembly variability reduces MS_E from 10.67 to 8.25, but the error df drop from 12 to 8, so the test is not necessarily more sensitive.

Balanced Incomplete Block Design (BIBD)

Sometimes a block cannot hold all treatments (batch size, a day holds only k runs, a car can test only some additives). In a **balanced incomplete block design**

- each block contains $k < a$ different treatments,
- every pair of treatments occurs together in exactly λ blocks, so all pairwise comparisons are estimated with the same precision.

Parameters: a treatments, b blocks, k treatments per block, r replicates of each treatment, $N = ar = bk$ runs, and

$$\lambda = \frac{r(k-1)}{a-1} \quad \text{must be an integer;} \qquad b \geq a \text{ (Fisher's inequality).}$$

If $a = b$ the design is **symmetric**. A BIBD always exists by taking all $\binom{a}{k}$ subsets as blocks, but that is often too many blocks; smaller BIBDs are tabulated (Montgomery, Appendix; Cochran and Cox, 1957).

Q: Four treatments in blocks of two: how many blocks do you need? (Problem 4.45)

BIBD - Intrablock Analysis

Model $y_{ij} = \mu + \tau_i + \beta_j + \varepsilon_{ij}$ for the observed cells. Treatments and blocks are **not orthogonal**, so the treatment SS must be **adjusted for blocks**:

$$SS_T = SS_{Treatments(adjusted)} + SS_{Blocks} + SS_E, \qquad SS_{Treatments(adjusted)} = \frac{k \sum_{i=1}^{a} Q_i^2}{\lambda a},$$

$$Q_i = y_{i.} - \frac{1}{k} \sum_{j=1}^{b} n_{ij}\, y_{.j}, \qquad n_{ij} = 1 \text{ if treatment } i \text{ is in block } j,\ 0 \text{ otherwise.}$$

Q_i is the total of treatment i minus the average of the blocks in which it appears.

Source	df	F_0
Treatments (adjusted)	$a - 1$	$MS_{Treatments(adj)} / MS_E$
Blocks (unadjusted)	$b - 1$	
Error	$N - a - b + 1$	
Total	$N - 1$	

In software: put the blocks **first** in a sequential (type I) ANOVA, or use type II sums of squares. Adjusted treatment estimates: $\hat{\tau}_i = kQ_i/(\lambda a)$, $\text{Var}(\hat{\tau}_i - \hat{\tau}_j) = 2k\sigma^2/(\lambda a)$.

BIBD - Catalyst Example

Montgomery, Example 4.5

The reaction time of a chemical process may depend on the catalyst. Batches of raw material are blocks, but each batch is large enough for three catalysts only; the order of the catalysts within a batch is randomized.

Balanced Incomplete Block Design for Catalyst Experiment

Treatment (Catalyst)	Block (Batch of Raw Material)				
	1	2	3	4	$y_{i.}$
1	73	74	—	71	218
2	—	75	67	72	214
3	73	75	68	—	216
4	75	—	72	75	222
$y_{.j}$	221	224	207	218	$870 = y_{..}$

$a = 4$ catalysts, $b = 4$ batches, $k = 3$, $r = 3$, $N = 12$, $\lambda = r(k-1)/(a-1) = 2$: each pair of catalysts appears together in two batches.

$$Q_1 = 218 - \tfrac{1}{3}(221 + 224 + 218) = -3, \quad Q_2 = -\tfrac{7}{3}, \quad Q_3 = -\tfrac{4}{3}, \quad Q_4 = \tfrac{20}{3},$$

$$SS_{Treatments(adjusted)} = \frac{3\,(9 + 49/9 + 16/9 + 400/9)}{2 \cdot 4} = 22.75.$$

BIBD - Catalyst Example in Python

```
cat = pd.DataFrame({
    "block": [1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4],
    "cat":   [1, 3, 4, 1, 2, 3, 2, 3, 4, 1, 2, 4],
    "y":  [73, 73, 75, 74, 75, 75, 67, 68, 72, 71, 72, 75]})
bibd = smf.ols("y ~ C(block) + C(cat)", data=cat).fit()
sm.stats.anova_lm(bibd, typ=1)           # blocks first
           df  sum_sq  mean_sq       F   PR(>F)
C(block)  3.0   55.00   18.333  28.205   0.0015
C(cat)    3.0   22.75    7.583  11.667   0.0107
Residual  5.0    3.25    0.650
```

The catalysts differ ($F_0 = 11.67$, $p = 0.011$). With the catalysts entered first, type I ANOVA gives the **unadjusted** catalyst SS 11.67 and the **adjusted** block SS 66.08 instead:

```
           df  sum_sq  mean_sq       F   PR(>F)
C(cat)    3.0   11.67    3.889   5.983   0.0415
C(block)  3.0   66.08   22.028  33.889   0.0010
Residual  5.0    3.25    0.650
```

The order of terms matters in a non-orthogonal design: always test treatments adjusted for blocks. `typ=2` gives both adjusted SS at once.

BIBD - Adjusted Means and Tukey's Test

Compare catalysts by the **adjusted** effects $\hat{\tau}_i = kQ_i/(\lambda a)$, not by the raw means:

```
k, lam, a = 3, 2, 4
yi = cat.groupby("cat").y.sum()                 # 218 214 216 222
yb = cat.groupby("block").y.sum()               # 221 224 207 218
Q = pd.Series({i: yi[i] - yb[d.block].sum() / k
               for i, d in cat.groupby("cat")})
# -3.000 -2.333 -1.333 6.667
tau = k * Q / (lam * a)                  # -1.125 -0.875 -0.500 2.500
q = stats.studentized_range.ppf(0.95, a, bibd.df_resid) # 5.218
hsd = q * np.sqrt(k * bibd.mse_resid / (lam * a))       # 2.58
```

Adjusted means $\bar{y}_{\cdot\cdot} + \hat{\tau}_i$: 71.375, 71.625, 72.000, 75.000.

pair	2-1	3-1	4-1	3-2	4-2	4-3
$\hat{\tau}_i - \hat{\tau}_j$	0.250	0.625	3.625	0.375	3.375	3.000
> 2.58?	no	no	yes	no	yes	yes

Catalyst 4 gives a longer reaction time than catalysts 1, 2 and 3, which do not differ. `pairwise_tukeyhsd()` would ignore the blocks and use the raw means.

Today Exercise

Ex04 – `code/01NAEX_Ex04.ipynb` (Python, Google Colab), a short exercise:

- The lecture examples in Python: rocket propellant (Latin and Graeco-Latin square), catalyst BIBD with adjusted means.
- Generating a randomized design and run sheet in Python (you will need it for the project).
- In-class problems from Montgomery: **4.23** (assembly methods, Latin square) and **4.40** (gasoline additives, BIBD); 4.42 as an optional extra.

There is **no separate homework** this week: the assignment is **Team Project 1**.

Next lecture (26 October): 2^k factorial designs (Montgomery, Chapter 6).

Team Project 1 - The Pendulum Experiment

Question: which factors affect the period T of a pendulum, and how well does the textbook formula work?

$$T_0 = 2\pi\sqrt{\frac{L}{g}} \quad \text{(small angles)}, \qquad T(\theta_0) \approx T_0\left(1 + \frac{\theta_0^2}{16} + \frac{11\,\theta_0^4}{3072}\right) \quad (\theta_0 \text{ in radians}).$$

Theory: T grows with $\sqrt{L}$, does not depend on the mass, and grows slowly with the release angle:

θ_0	10°	20°	30°	45°	50°	60°
$T/T_0 - 1$	0.19 %	0.77 %	1.74 %	4.0 %	5.0 %	7.3 %

For $L = 0.65\,\text{m}$ ($T_0 = 1.62\,\text{s}$) the step from 10° to 30° is only 0.025 s. On the T scale the effects of L and θ_0 multiply ($L \times \theta_0$ interaction), on the $\log T$ scale they add.

Last year's lesson: with angles of 5–20° and a stopwatch the angle effect was invisible, and *"mass is not significant"* was reported as a proof that mass does not matter.

This year you have to **plan the precision** so that the angle effect becomes detectable, and you have to show that mass does not matter with an **equivalence test**, not with a large p-value.

Team Project 1 - Design Requirements

- **Setup:** three pendulums hung in advance at three lengths L (pivot to the centre of the ring, measured); mass = 1, 2 or 3 identical rings threaded at one point (weighed); release angle θ_0: 3 levels, at least one $\leq 10^\circ$ and one $\geq 45^\circ$.
- **Response:** period $T = t_N/N$ from the time t_N of $N = 20$ full oscillations; record the raw t_N. One operator times each run.
- **Pilot study** (10 runs at one setting) $\Rightarrow \hat{\sigma} \Rightarrow$ number of blocks for power ≥ 0.9 to detect the 10° vs 30° difference (noncentral F, Lecture 3).
- **Part A – RCBD:** the 3×3 factorial $L \times \theta_0$ (mass fixed), block = operator, at least 3 blocks, random order within each block.
- **Part B – two** 3×3 **Graeco-Latin squares:** rows = operator, columns = mass, Latin letters = angle, Greek letters = length; each square randomized separately.
- About 55 runs, i.e. about 2 hours. Record the run order, operator, time and any problems (a missed count, a twisted string, . . .).

Team Project 1 - Analysis and Submission

Analysis (details in `projects/01NAEX_Project1.ipynb`):

- Part A: factorial ANOVA in blocks for T and for $\log T$; why does the $L \times \theta_0$ interaction depend on the scale? Tukey with the correct MS_E; relative efficiency of blocking.
- Part B: Graeco-Latin ANOVA; why does one square leave no error df, and why does the $L \times \theta_0$ interaction on the T scale show up as a false mass or operator effect (aliasing)?
- Numeric model: $\log T$ on $\log L$, the angle correction $\log(1 + \theta_0^2/16 + \dots)$ and the measured mass; exponent of L (theory $1/2$) and g with a confidence interval.
- Mass: confidence interval and an equivalence test (TOST) with a margin you justify.
- Residual diagnostics (also versus run order and operator); comparison of the two designs per run.

Teams of 2–4 students. **Submit** by pull request into `projects/`: notebook `01NAEX_Project1_<Surname1>_<Surname2>....ipynb` with outputs and the raw data as a CSV file of the same name.

Deadline: Sunday 8 November 2026.